Great question — setting up a **MERN Stack backend** (Node + Express + MongoDB) requires a few essential files and folders to get a clean, scalable foundation.

Here's a complete list of the **basic files and folders** you'll need to set up the backend (Express + MongoDB + Mongoose + TypeScript or JavaScript):

✅ 1. Folder Structure

```
MERN-Backend/
├── src/
│   ├── config/
│   │   └── db.config.ts
│   ├── controllers/
│   │   └── user.controller.ts
│   ├── models/
│   │   └── user.model.ts
│   ├── routes/
│   │   └── user.routes.ts
│   ├── middlewares/
│   │   └── errorHandler.ts
│   ├── utils/
│   │   └── helpers.ts (optional)
│   └── server.ts
├── .env
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.json (if using TypeScript)
└── README.md
```

📄 Essential File Explanations

🔹 `src/server.ts`

Main entry point. Boots up Express and connects MongoDB.

```
import express from 'express';
import dotenv from 'dotenv';
import { ConnectDatabase } from './config/db.config';
import userRoutes from './routes/user.routes';
import { errorHandler } from './middlewares/errorHandler';
```

```
dotenv.config();
const app = express();
const PORT = process.env.PORT || 5000;

ConnectDatabase(process.env.DB_URI!);

app.use(express.json());
app.use('/api/users', userRoutes);

// Error Handler Middleware
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

🔹 src/config/db.config.ts

MongoDB connection logic.

```
import mongoose from 'mongoose';

export const ConnectDatabase = async (uri: string) => {
  try {
    await mongoose.connect(uri);
    console.log('✅ MongoDB Connected');
  } catch (error) {
    console.error('❌ MongoDB connection error:', error);
    process.exit(1);
  }
};
```

🔹 src/models/user.model.ts

Defines Mongoose schema.

```
import mongoose from 'mongoose';

const UserSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
}, { timestamps: true });
```

```
export const User = mongoose.model('User', UserSchema);
```

🔹 src/routes/user.routes.ts

Defines Express routes.

```
import express from 'express';
import { getUsers } from '../controllers/user.controller';

const router = express.Router();

router.get('/', getUsers);

export default router;
```

🔹 src/controllers/user.controller.ts

Route logic (controller).

```
import { Request, Response } from 'express';

export const getUsers = async (req: Request, res: Response) => {
  res.status(200).json({ message: 'All users' });
};
```

🔹 src/middlewares/errorHandler.ts

Central error handling middleware.

```
import { Request, Response, NextFunction } from 'express';

export const errorHandler = (err: any, req: Request, res: Response, next: NextFunction) => {
  const status = err.statusCode || 500;
  res.status(status).json({
    success: false,
    message: err.message || 'Server Error',
  });
};
```

🔹 `.env`

Environment variables (not pushed to GitHub)

```
PORT=8080
DB_URI=mongodb://localhost:27017/mernproject
```

🔹 `.env.example`

A sample template of `.env` for others:

```
PORT=
DB_URI=
```

🔹 `.gitignore`

To ignore sensitive or unnecessary files.

```
node_modules/
.env
dist/
```

🔹 `package.json`

Basic dependencies (install with `npm init -y`)

If using TypeScript:

```
{
  "scripts": {
    "dev": "ts-node-dev src/server.ts"
  },
  "dependencies": {
    "express": "^4.18.2",
    "dotenv": "^17.2.0",
    "mongoose": "^8.0.0"
  },
  "devDependencies": {
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.0.0",
    "@types/node": "^20.0.0",
    "@types/express": "^4.17.0"
```

```
  }
}
```

Install with:

```
npm install express mongoose dotenv
npm install -D typescript ts-node-dev @types/node @types/express
```

🔹 `tsconfig.json` (if TypeScript)

Basic TS setup.

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  }
}
```

✅ Summary: Bare Minimum Required Files

Type	File	Purpose
Entry Point	`src/server.ts`	Start server & mount routes
Config	`src/config/db.config.ts`	Connect MongoDB
Routes	`src/routes/user.routes.ts`	Define API endpoints
Controller	`src/controllers/user.controller.ts`	Route logic
Model	`src/models/user.model.ts`	MongoDB schema
Middleware	`src/middlewares/errorHandler.ts`	Handle errors globally
Env	`.env` / `.env.example`	Store sensitive configs

Git	`.gitignore`	Ignore node_modules, env, etc.
TypeScript	`tsconfig.json`	Compiler config
Dependencies	`package.json`	Manage project dependencies

Would you like me to generate a zip/starter project with this entire boilerplate? Or do you want a JavaScript (non-TypeScript) version instead?